淡 江 大 學 電 機 工 程 學 系

專 題 實 驗 報 告

Post Quantum Cryptography: Kyber Encryption Hardware Implementation

指導教授：施鴻源 博士

專 題 生：陳思妤

中 華 民 國 115 年 2 月

Abstract

CRYSTALS-KYBER (ML-KEM) is a standardized post-quantum public-key cryptosystem designed to replace classical schemes that are vulnerable to quantum adversaries under Shor's algorithm. This work presents a complete hardware implementation of the Kyber CPA-secure public-key encryption (PKE) primitive, focusing on modularity, verifiability, and architectural clarity. The design includes synthesizable modules for Number Theoretic Transform (NTT and InvNTT), modular reduction, Centered Binomial Noise Sampling, polynomial point-wise multiplication, decompression, compression, and a controller-based coordination framework. Hash functions required by KYBER are emulated through software to simplify development while preserving hardware-compatible interfaces.

The proposed architecture emphasizes easy debugging, deterministic module interaction, and compatibility with future hardware–software co-design and side-channel evaluation. Functional correctness is verified against the official reference implementation using Python-based test vector generation and simulation. This hardware serves as a lightweight, fully synthesizable foundation for future exploration of performance optimization and security hardening in post-quantum cryptographic accelerators.

Table of Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1 Introduction

1.1 Background

Post Quantum Cryptography (PQC) is driven by the fact that classical public-key is no longer secure under the attack of quantum computers using Shor's Algorithm, which can solve the Integer Factorization Problem in polynomial time.

CRYSTALS-KYBER is a module-lattice based public-key cryptosystem whose security is based on the hardness of Module-LWE (Learning-With-Errors) problem [1]. Even quantum adversary would not be able to solve the worst-case lattice problem in polynomial time.

In August 2024, KYBER is standardized as Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) in Federal Information Processing Standards Publication 203 (FIPS203) by National Institute of Standards and Technology (NIST) [2]. While the software implementation is well-studied, hardware is usually stronger in calculation efficiency. Furthermore, there already exist hardware research optimizing the throughput or speed. However, there is limited availability of lightweight, fully synthesizable hardware implementations that can serve as a practical testbed for security evaluation.

This work addresses the gap by developing a complete Verilog implementation of Kyber's public-key encryption (PKE) scheme intended for architectural exploration, future hardware-based security assessment, and the evaluation of potential side-channel countermeasures.

1.2 Contribution

1. Developed a fully modular and testable hardware design of individual modules, available on Github: `https://github.com/Astelor/project-kyber`.
2. Developed a handshake architecture to improve flexibility of the inter-module timing and facilitate resource reuse.
3. Proposed a hardware and software co-design framework for module verification and development.

1.3 Structure of this report

Chapter 1 gives a brief introduction on Kyber and this work. Chapter 2 lists the development environment for this work, Chapter 3 gives an overview on Kyber PKE encryption algorithm. Chapter 4 includes more detailed report on the implemented hardware. Last but not least, Chapter 5 discusses the design choices and trade-offs.

Chapter 2 Development Environment

2.1 Quartus II

Quartus II is an integrated development environment (IDE) provided by Intel (formerly Altera) for designing, simulating, and implementing digital systems on FPGA devices. It offers a complete tool chain that supports phases of hardware development. In this work, Quartus II is primarily used for synthesizing the Verilog hardware implementation and for verifying functional behavior through ModelSim. These tools provide the essential workflow for translating HDL designs into synthesizable logic and confirming correctness of the hardware modules.

2.2 Experiment Environment

The experiment environment is listed in Table 2.1. Hardware synthesis and behavioral verification is done using Quartus II and ModelSim. Verilog is used for HDL programming while Python and C are used for software level functional verification and data generation. ModelSim commands are used in testbenches and simulation scripts to help with debugging.

Operating System	Windows 11
CPU	Intel i5-1235U
Installed RAM	24GB
Software	Quartus II, ModelSim
Prog. language	Verilog, Python, C, ModelSim Command

Table 2.1: Development environment

Chapter 3 Introduction to KYBER

3.1 Overview of KYBER

The standardized ML-KEM is proposed as two parts, with security of indistinguishability under chosen-ciphertext and chosen-plaintext attacks (IND-CCA and IND-CPA) respectively, these structures are directly mapped to the original CRYSTALS-KYBER paper published in 2017.

The CCA secure part utilizes Fujisaki–Okamoto transform which implicitly rejects malformed ciphertext in the decapsulation process, allowing the construction of a KEM. The CPA secure part is the public-key encryption primitive of this cryptosystem, namely the subject of this work, KYBER.

$$\hat{\mathbf{t}} = \hat{\mathbf{A}} \circ \hat{\mathbf{s}} + \hat{\mathbf{e}} \tag{3.1}$$

Encryption key-pair and decryption key are $(\hat{\mathbf{t}}, \hat{\mathbf{A}})$ and $\hat{\mathbf{s}}$ respectively. The relationship of these keys is denoted in Equation 3.1. Note that $\hat{\mathbf{e}}$ is the sampled noise.

KYBER PKE system can be defined in as $\text{decrypt}(dk_{PKE}, \text{encrypt}(ek_{PKE}, A, m)) = m$ where m is the 256 bit symmetric key for AES, the correctness relies on the fact that noise terms introduced during encryption stay within a bounded range, so after compressing, the reconciliation mechanism recovers the original message bits. See [1, Section 3] for more details.

ML-KEM provides three parameter sets as listed in Table 3.1 for different security strengths that balances between communication size and security. Due to the project's time constraint, only ML-KEM-768 and PKE Encryption is implemented.

	n	q	k	η_1	η_2	d_u	d_v
ML-KEM-512	256	3329	2	3	2	10	4
ML-KEM-768	256	3329	3	2	2	10	4
ML-KEM-1024	256	3329	4	2	2	11	5

Table 3.1: ML-KEM parameter set

3.2 KYBER Encryption

The following section is a brief explanation of KYBER encryption shown in Algorithm 1.

There are four necessary inputs: the encryption key, matrix generation seed, message to be used as the secret key, and the randomness intended for noise term generation. The matrix $\hat{\mathbf{A}}$ is regenerated during encryption process to reduce the key size.

Since the encryption key is encoded in the key generation process to eliminate redundant bits and produce more compact data, it will be decoded into integers for later calculations. The message encode and decompress process is to upscale the bits, by serializing the integer and multiplying the individual zeros and ones by $\lceil \frac{q}{2} \rfloor$ so that the message can be uncovered for decryption.

There are three noise terms: $\mathbf{y}$, $\mathbf{e_1}$, and e_2. The first one is the main LWE noise term, and others are to increase the difficulty of solving LWE. $\mathbf{y}$, is transformed to NTT domain to perform point-wise multiplication with the encryption keys. The products are transformed back to normal domain by inverse NTT to ensure the correctness of addition operation for $\mathbf{e_1}$ and e_2. And then the pre-ciphertexts, $\mathbf{u}$ and v, are compressed and encoded to reduce data size without affecting the correctness. Eventually the ciphertexts are concatenated and outputted as c. See the Table 3.1 for the parameters such as k.

To put the data types into perspective, polynomials of degree 255 can be think of as an array of integers modulo q denoted as $\mathbb{Z}_q^{256}$. Vectors can be think of as an array of $\mathbb{Z}_q^{256}$. Matrices can be think of as an array of vectors, that is k in depth. The notation of "hat" means the polynomial is transformed to NTT domain, which allows the point-wise multiplication of two polynomials, reducing the computational complexity from $O(256^2)$ to $O(256 \log 256)$

Algorithm 1 Kyber.CPA.Encryption

Input: encryption key $\text{ek}_{\text{PKE}} \in 384k$ Bytes
Input: matrix generation seed $\rho \in 32$ Bytes
Input: message $m \in 32$ Bytes
Input: randomness $r \in 32$ Bytes ▷ For hash digest for CBD sampling
Output: ciphertext $c \in (32(d_u k + d_v))$ Bytes

$N \leftarrow 0$
$\hat{t} \leftarrow \text{Decode}_{12}(\text{ek}_{\text{PKE}}[0 : 384k])$
$\mu \leftarrow \text{Decompress}_1(\text{Encode}_1(m))$
for $(i \leftarrow 0; i < k; i{+}{+})$ **do**
    for $(j \leftarrow 0; j < k; j{+}{+})$ **do**
        $\hat{\mathbf{A}}[i, j] \leftarrow \text{SampleNTT}(\rho||j||i)$
    end for
end for
for $(i \leftarrow 0; i < k; i{+}{+})$ **do**
    $\mathbf{y}[i] \leftarrow \text{CBD}_{\eta_1}(\text{SHAKE256}(r||N, 8 \cdot 64 \cdot \eta_1))$
    $N \leftarrow N + 1$
end for
for $(i \leftarrow 0; i < k; i{+}{+})$ **do**
    $\mathbf{e_1}[i] \leftarrow \text{CBD}_{\eta_2}(\text{SHAKE256}(r||N, 8 \cdot 64 \cdot \eta_2))$
    $N \leftarrow N + 1$
end for
$e_2[i] \leftarrow \text{CBD}_{\eta_2}(\text{SHAKE256}(r||N, 8 \cdot 64 \cdot \eta_2))$
$\hat{\mathbf{y}} \leftarrow \text{NTT}(\mathbf{y})$ ▷ run NTT k times
$v \leftarrow \text{NTT}^{-1}(\hat{\mathbf{t}} \circ \hat{\mathbf{y}}) + e_2 + \mu$
$\mathbf{u} \leftarrow \text{NTT}^{-1}(\hat{\mathbf{A}} \circ \hat{\mathbf{y}}) + \mathbf{e_1}$ ▷ run NTT^{-1} k times
$c_2 \leftarrow \text{Encode}_{d_v}(\text{Compress}_{d_v}(v))$
$c_1 \leftarrow \text{Encode}_{d_u}(\text{Compress}_{d_u}(\mathbf{u}))$
return $c \leftarrow (c_1||c_2)$

Chapter 4 Hardware Implementation

4.1 Methodologies

The mathematical foundation is composed of the FIPS203 specification. Moreover, the algorithmic and modular design of this work is heavily based on the C reference implementation by the PQ-CRYSTALS team [3]. In particular, the hardware modules are procedurally tested against the reference implementation using basic test vectors to verify the correctness of modules, so as to facilitate the higher-level integration.

4.2 Architecture

The high level architecture of KYBER Encryption is illustrated in Figure 4.1. It is a simple approach to address the necessary modules for encryption. The four input data are fed into the module via testbench, computed using Algorithm 1, and outputted to an register array in the testbench.

The hash modules are described in Section 4.3, CBD is described in Section 4.4, the reduction modules are described in Section 4.5, NTT and Inverse NTT (InvNTT) are described in Section 4.6, Vector Point-wise Multiplication is described in Section 4.7, Decompress is described in Section 4.8, Accumulators are described in Section 4.10.

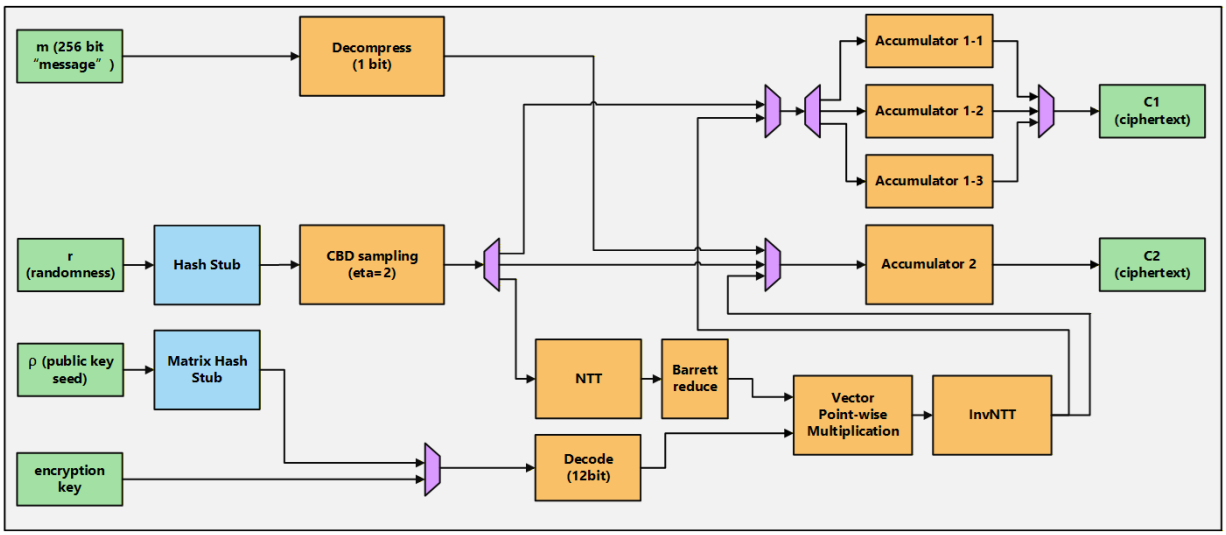


Figure 4.1: Architecture of KYBER encryption hardware implementation

The design philosophy is to use controllers for all the functional modules in Figure 4.1 so that they don't directly "talk" with each other, we refer to this type of controller as "Small Controller."

The controllers talk with each other via their status codes and each controllers will act according to the status codes they received and the current state they are in. Since the encryption scheme is fixed, we design a ”Big Controller” that issue codes to the Small Controllers for coherent coordination. This marks distinction for control hierarchy so that no signal conflict can occur, at the cost of negligible latency. A part of the controller design is illustrated in Figure 4.2.

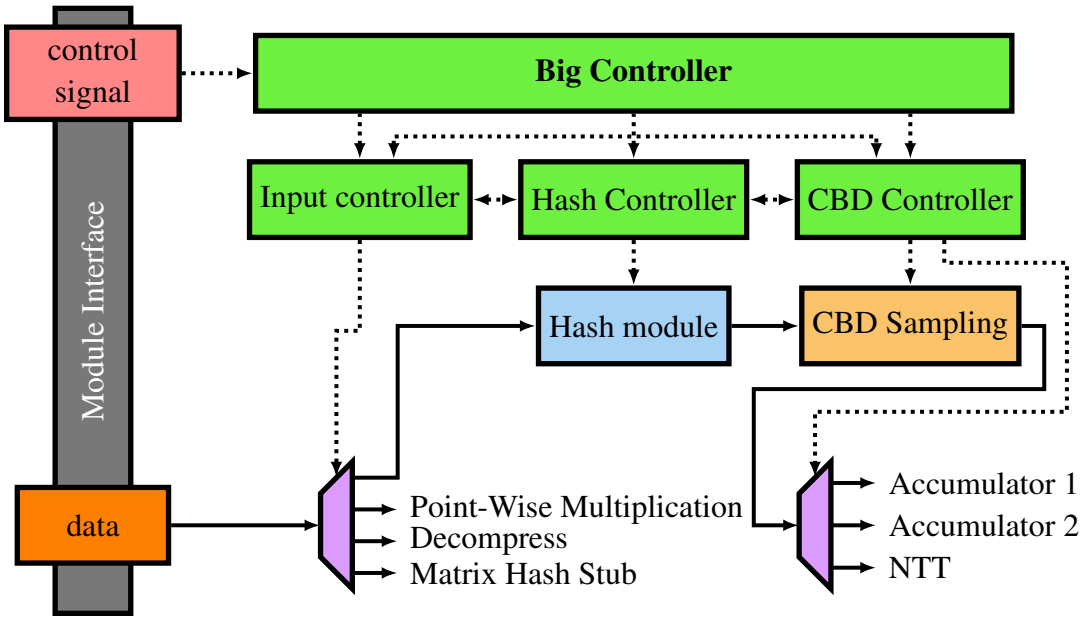


Figure 4.2: Module controller design architecture snippet

The Small Controllers only receive status code from neighboring ones, such that InvNTT never needs to know what Hash Stub is doing. To give an example on a line of control: CBD need to receive data from Hash Stub and transport data to NTT or Accumulator 1 or Accumulator 2. So the Small Controller for CBD need to receive status code from the previously mentioned modules' Small Controllers and send its own status code to them.

The finite state machine (FSM) concept is shown in Figure 4.3. Specifically, the inter-controller status code is essentially the controller's own FSM state.

The concept works as follows: initially, after reset, the FSM enters the `IDLE` state. Once the module completes initialization, the FSM transitions to `CHOOSE` and chooses which state to go to based on the command from the Big Controller. Assuming we transition to `INPUT READY`, which is dependent on the status code `output ready` from the other Small Controllers. When it receives the code, it will transition to `INPUT`. This constructs a hand-shake with the sender since it will also get the receiver's status code, that is, `input ready`. Once the sender completes output, it will send `output done`, then the machine transitions to `START CAL`. This is a intermediate state before `CALCULATE` to make sure the flags that are switched during input are switched back to normal, so there is no condition on the transition. Upon module calculation completion, the module will send `done` and the machine will transition to `OUTPUT READY`, to initiate the hand-shake with another receiver. However, the `OUTPUT` terminating condition is from a counter in the top-level module and the `<number>` is fixed in the FSM. The `OUTPUT DONE` state will transition to `SEQUENCE DONE` in order to reset the flags and wait for Big Controller to issue a reset command and the machine will go back to `IDLE`.

Note that each controller's FSM is adjusted according to its own module's needs, but they are designed based on the concept.

The modules can be seen as individual servers containing buffers for their inputted and output data. They need to rely on their own controllers to tell them what to do, since the modules cannot sense the "outside world". Such inter-module coordination via controllers is critical to ensure the correctness of parallelism and data transportation. Figure 4.4 shows the parallelism and data transportation in high level abstraction without accounting for computation latency. The text inside of the box denotes it's output data (also the result of its computation) and the order of data input and output is accurate to the hardware implementation.

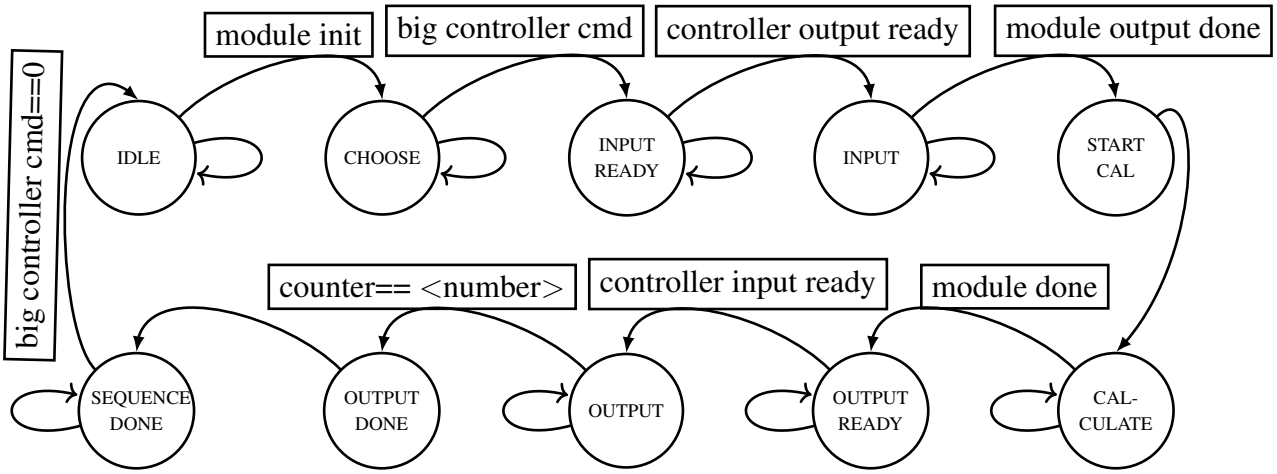


Figure 4.3: State machine concept for Small Controllers

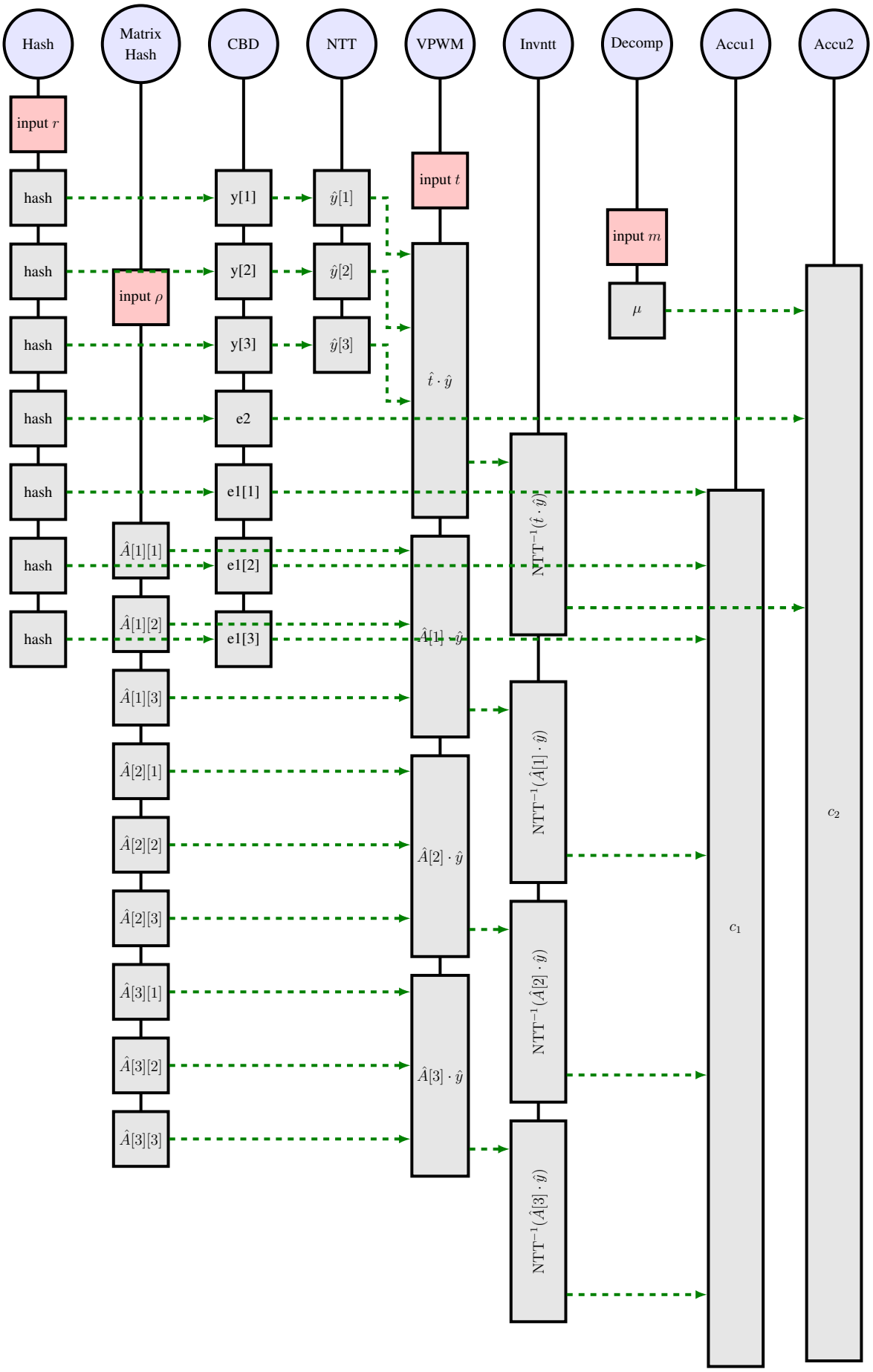


Figure 4.4: Module parallel computation and data transportation timing

4.3 Emulated Hash Modules

To focus on the architectural development of Kyber, Extendable-output functions in SHA-3, namely SHAKE128 and SHAKE256 [4], are emulated with not synthesizable Python and ModelSim system tasks.

The emulated modules are Hash Stub and Matrix Hash Stub modules in Figure 4.1, their respective functions are SHAKE256 and SampleNTT in Algorithm 1. We preserves synthesizable hardware interface while the behavioral function is achieved by software, so that the hardware hash implementations can be swapped in with minimal disruption in the future.

The hardware and software implementation communicate their statuses and data transfers via in the development environment, those written by hardware can only be read by software, and vice versa.

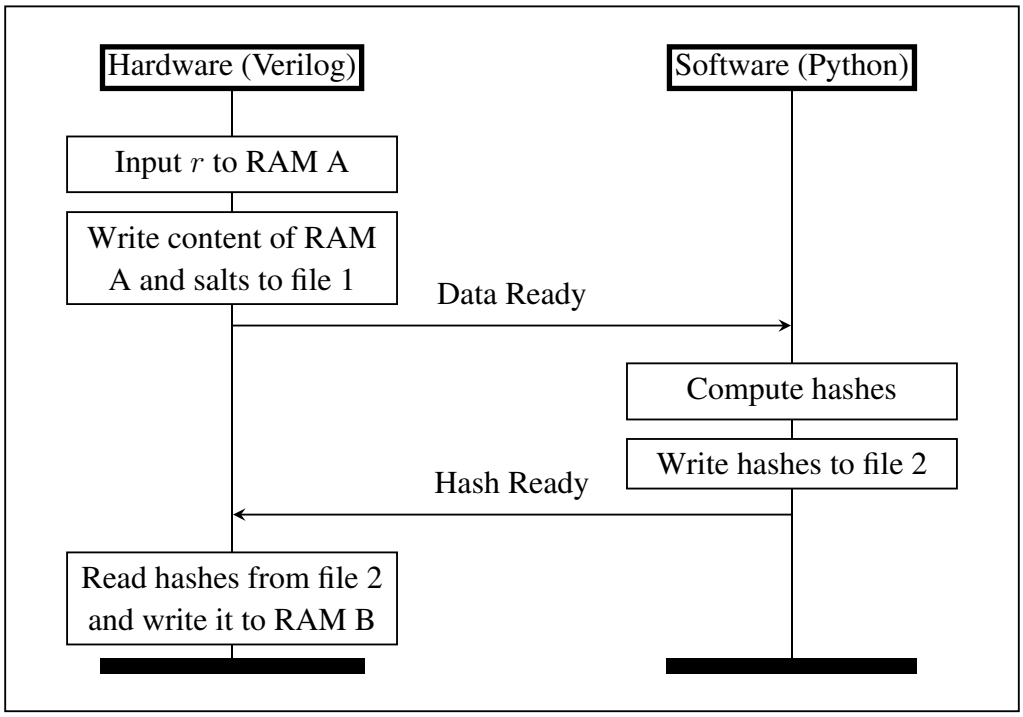


Figure 4.5: Hash stub and Verilog modules interaction via files

4.4 Noise Sampling

The noise polynomials are sampled from Centered Binomial Distribution (CBD), specifically using an array of uniformly random bits $(b_1, ..., b_{64 \cdot 8\eta})$ and output the error polynomial $(e_1, ..., e_{256})$ by computing $e_i = \sum_{j=1}^{\eta} b_{2(i-1)\eta+j} - \sum_{j=1}^{\eta} b_{2(i-1)\eta+\eta+j}$.

Algorithm 2 takes a 32-bit string to be sampled into 8 integers $\in [-2, +2]$ and encoded into a 32 bit string in little-endian by bytes. This way it conforms to the RAM block data size of 32 bit and increases the compactness of storage.

The encoded numbers are decoded into 16 bit signed integers during the output of CBD module, which is illustrated in Figure 4.6. The timing accounts for the address increment and RAM read latency to make the coefficient output as seamless as possible. Note that the number in coefficient output timing graph denotes index of the encoded array.

Algorithm 2 CBD_2 sampling and encoding

Input: random byte array $b \in 4\,\mathrm{Bytes}$
Output: encoded error array $f \in 4\,\mathrm{Bytes}$

1: $d \leftarrow b\,\&\,0\mathrm{x}55555555 + (b >> 1)\,\&\,0\mathrm{x}55555555$ ▷ "&" is bitwise AND operation
2: **for** $(i \leftarrow 0; i < 8; i{+}{+})$ **do**
3: $\quad a \leftarrow ((d >> 4i)\,\&\,3 - (d >> (4i+2))\,\&\,3)\,\&\,0\mathrm{xF}$ ▷ ">>" is logical right shift
4: $\quad f \leftarrow f||(a << (24 - 8(i >> 1) + 4(i\,\&\,1)))$ ▷ "<<" is logical left shift
5: **end for**
6: Return f

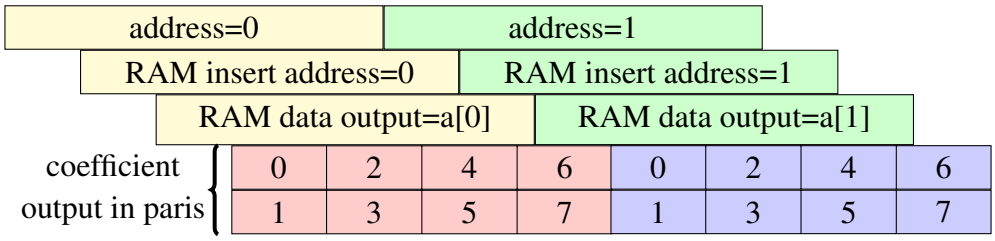


Figure 4.6: CBD output decode timing

The Matrix Hash Stub that generates one of the encryption key uses the derived functions of SHAKE128 defined in [5], which can be think of as the internal API of SHAKE128. The key generation function performs rejection sampling based on the bytes produced by the SHAKE128.Squeeze function. Since it requires Hash functions that is not in the scope of this work, it is done by the method described in Section 4.3. The full algorithm can be found in [2, Section 4.1].

4.5 Modular Reduction

There are two types of modular reduction algorithms in this work, Montgomery reduction [6] and Barrett reduction [7]. The Algorithm 3 and 4 are the same as those used in the software implementation [3].

Montgomery reduction reduces 32-bit signed integers, effectively calculating congruence of $aR^{-1} \mod q$ with $a \in \mathbb{Z}$ and $R = 2^{16}$. For single Montgomery reduction, the results will need to be converted from Montgomery form to normal form. However, in a chain of multiplications, the intermediate value can remain in Montgomery form and eventually be converted in a comparatively small time.

Algorithm 3 32-bit Montgomery Reduction

Input: 32-bit signed integer a
Output: 16-bit signed integer t
1: $m \leftarrow a \cdot q^{-1} \mod 2^{16}$
2: $t \leftarrow (a - m \cdot 3329) \div 2^{16}$
3: Return t

Barrett reduction is used for single reductions to avoid the overhead of form conversion in Montgomery reduction. The reduction hardware are written in similar fashion to the algorithms and initiated in places required, as they are not reused among the functional modules.

Algorithm 4 16-bit Barrett Reduction

Input: 16-bit integer a
Output: 16-bit integer t
1: $m \leftarrow a \cdot 20159 + 2^{25}$ ▷ $(2^{26} + \frac{q}{2}) \div q \simeq 20159$
2: $m \leftarrow \frac{m}{2^{26}}$ ▷ Done by left arithmetic shift
3: $t \leftarrow a - m \cdot q$
4: Return t

Since Montgomery reduction is used for reducing products, which is used for NTT/InvNTT in Section 4.6 and point-wise multiplication in NTT domain in Section 4.7. The result of such reduction is in Montgomery form, effectively scaling $a \mod q$ to $a \cdot 2^{-16} \mod q$. Informally, by basic modular arithmetic, addition is closed under Montgomery form and multiplication is not. To ensure the correctness on NTT and InvNTT, the ζ values are scaled by $-1044 \equiv 2^{16} \mod q$ so that the products remain in normal form.

However, point-wise multiplication results in Montgomery form, effectively we need to multiply two $2^{16} \mod q$ with the coefficients to convert from Montgomery form to normal form using the Montgomery reduction, this is done in the last state of InvNTT in Algorithm 6.

4.6 Number Theoretic Transform

Number Theoretic Transform (NTT) is a specialized version of Discrete Fourier Transform used to improve the efficiency of multiplication in ring R_q. The ring R_q is isomorphic to another ring T_q denoted in Equation 4.1. NTT/InvNTT is a computationally efficient algorithm that converts between these two rings.

Note that the function $\mathrm{BitRev}_7(i)$ returns the bit-reversed unsigned 7-bit input integer $i \in 0, 1, ..., 127$ and $\zeta = 17$.

$$R_q := \mathbb{Z}_q[X]/(X^{256}+1) \qquad T_q := \bigoplus_{i=0}^{127} \mathbb{Z}_q[X]/(X^2 - \zeta^{2\mathrm{BitRev}_7(i)+1}) \tag{4.1}$$

$$f = f_0 + f_1 X + f_2 X^2 + \cdots + f_{255} X^{255} \in R_q \tag{4.2}$$

$$\hat{f} = (\hat{f}_0 + \hat{f}_1 X) + \cdots + (\hat{f}_{254} + \hat{f}_{255} X) \in T_q \tag{4.3}$$

NTT uses Cooley-Tukey [8] algorithm and InvNTT uses Gentleman-Sandle [9] algorithm. Both algorithms are conforming to FIPS203 as denoted in Algorithm 5 and 6, Note that Montgomery reduction is used for both NTT and InvNTT.

For hardware implementation, only one butterfly operation is used due to resource and memory access constraints. Therefore, the hardware is more or less a pipelined sequential form of its implemented algorithm. The butterfly operations are illustrated in Figure 4.8a and 4.8b.

The for-loops are unrolled to read coefficients from memory, perform butterfly operations, and write the results back to memory. This is done by "ping-pong" between two RAM blocks in Figure 4.7, we will refer to this structure as ping-pong RAM. Note that the RAM blocks are dual-port RAM, which allow two unique addresses performing either read or write action simultaneously.

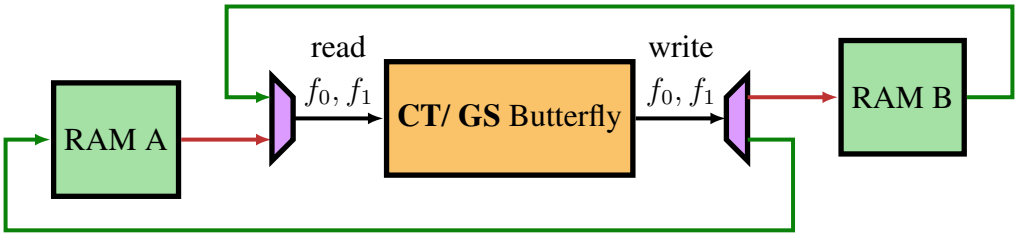


Figure 4.7: NTT ping-pong RAM architecture

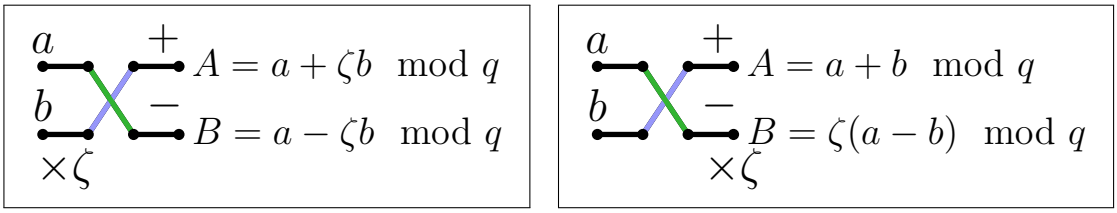
The hardware butterfly is shown in Figure 4.9a and 4.9b. Both NTT and InvNTT use the same addressing architecture in Figure 4.10, this software-like unrolled for-loop uses simple arithmetic and bitwise operation to detect boundaries of loops in Algorithm 5. The left-most decision block can be mapped to the first for-loop. The middle decision block act as the second and third for-loop. The right-most decision block can be mapped to the stop or increment condition for the first for-loop. Note that for NTT and InvNTT the len is initialized as 128 and 2 respectively. And for InvNTT right shift operation on len is changed to left shift.

The read and write index are kept separately for ping-ping RAM to address memory read and computation latency. Its timing is illustrated in Figure 4.7. RAM A and RAM B read/write is controlled by the ”layer” variable that is initialized as 1 and increments once ”RAM write” is completed. The timer decides when ”RAM write” starts. Timer starts when the read flag is switched to 1 and stops when it reaches a fixed value, then switches the write flag to 1. This way it allows pipelining and prevents RAM read/write conflict.

The ping-pong RAM works as follows: initially, the input polynomial is written to RAM A. During the computation, it will first read two coefficients (at address i and i+len) from RAM A, perform the butterfly operation, and write the results to RAM B (at address wr_i and wr_i+wr_len). For the next layer, which is an even number, the data will be read from RAM B and write the results to RAM A. The read and write role for RAM A and RAM B are switched repeatedly according to the layer until the algorithm is done, hence the nickname ”ping-pong”.

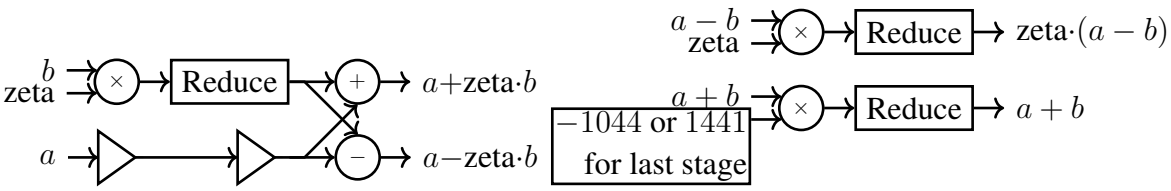
To address the extra operation on all coefficients in the end of InvNTT, we upscale the last zeta by $-1044/128 \mod q$. Effectively converting the polynomial from Montgomery domain to normal domain while multiplying $1/128 \mod q$ in the mean time. Note that the zetas are already upscaled by $-1044 \equiv 2^{16} \mod q$.

The zetas are pre-computed and stored in a ROM, both NTT and InvNTT modules have their own zeta ROM instance to lower the design complexity.



(a) Cooley-Tukey (CT) Butterfly (b) Gentelman-Sande (GS) Butterfly

Figure 4.8: Butterfly operations for NTT and InvNTT



(a) Cooley-Tukey (CT) Butterfly hardware (b) Gentelman-Sande (GS) Butterfly hardware

Figure 4.9: CT and GS butterfly hardware

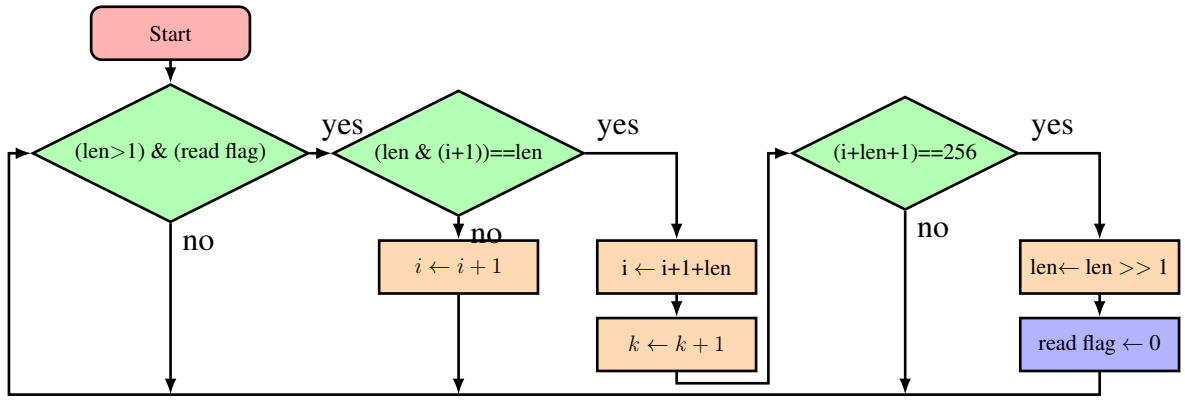


Figure 4.10: NTT read index generation

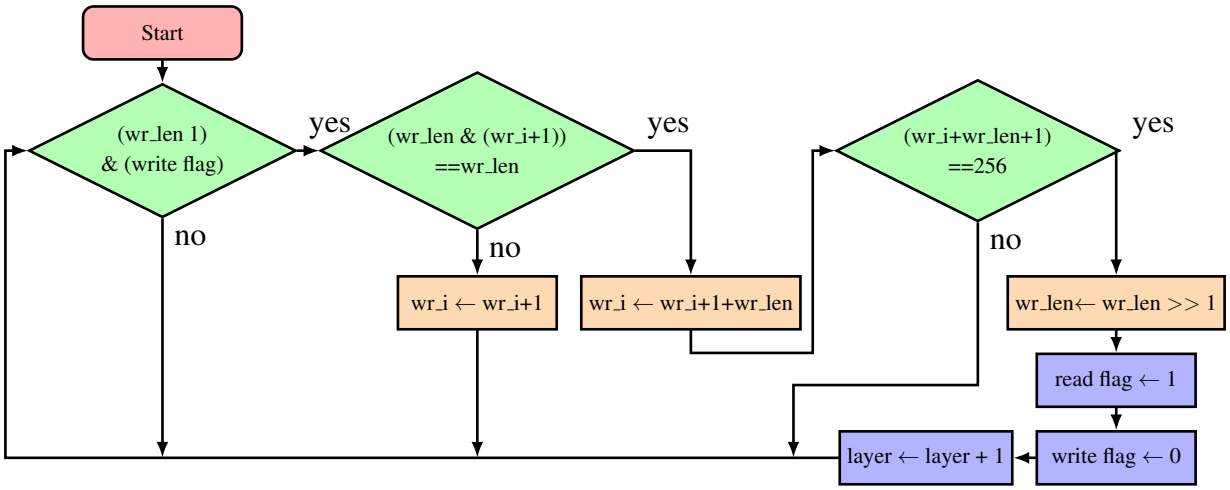


Figure 4.11: NTT write index generation

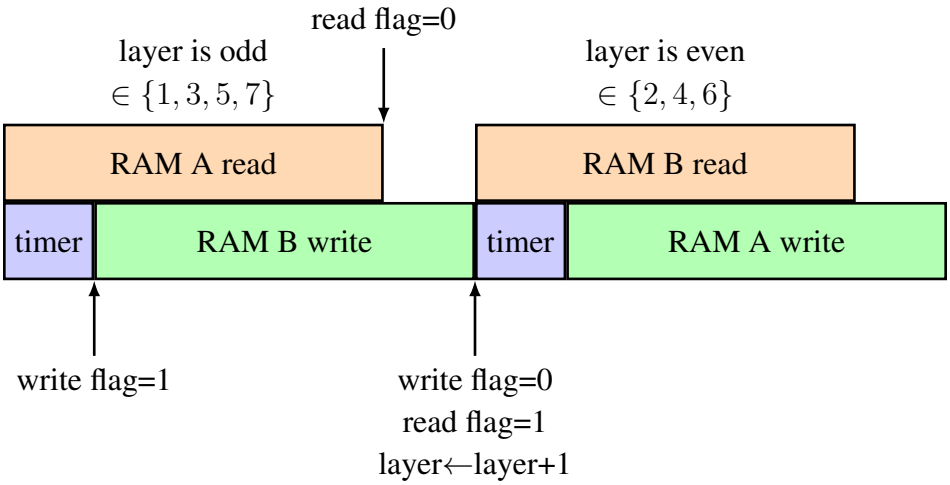


Figure 4.12: NTT ping-pong RAM timing

Algorithm 5 NTT algorithm

Input: polynomial array $f \in \mathbb{Z}_q^{256}$
Output: polynomial array $\hat{f} \in \mathbb{Z}_q^{256}$

$\hat{f} \leftarrow f$
$i \leftarrow 1$
for ($len \leftarrow 128$; $len \geq 2$; $len \leftarrow len/2$) **do**
  for ($start \leftarrow 0$; $start < 256$; $start \leftarrow start + 2 \cdot len$) **do**
    $zeta \leftarrow \zeta^{\mathrm{BitRev}_7(i)} \mod q$
    $i \leftarrow i + 1$
    for ($j \leftarrow start$; $j \ll start + len$; $j++$) **do**
      $t \leftarrow zeta \cdot \hat{f}[j + len]$ ▷ CT Butterfly
      $\hat{f}[j + len] \leftarrow \hat{f}[j] - t$
      $\hat{f}[j] \leftarrow \hat{f}[j] + t$
    end for
  end for
end for
Return $\hat{f}$

Algorithm 6 InvNTT algorithm

Input: polynomial array $\hat{f} \in \mathbb{Z}_q^{256}$
Output: polynomial array $f \in \mathbb{Z}_q^{256}$

$f \leftarrow \hat{f}$
$i \leftarrow 127$
for ($len \leftarrow 2$; $len \leq 128$; $len \leftarrow 2 \cdot len$) **do**
  for ($start \leftarrow 0$; $start < 256$; $start \leftarrow start + 2 \cdot len$) **do**
    $zeta \leftarrow \zeta^{\mathrm{BitRev}_7(i)} \mod q$
    $i \leftarrow i - 1$
    for ($j \leftarrow start$; $j \ll start + len$; $j++$) **do**
      $t \leftarrow f[j]$ ▷ GS Butterfly
      $f[j] \leftarrow t + f[j + len]$
      $f[j + len] \leftarrow zeta \cdot (f[j + len] - t)$
    end for
  end for
end for
$f \leftarrow f \cdot 3303 \mod q$ ▷ $3303 \equiv 128^{-1} \mod q$
Return f

4.7 Point-wise Multiplication

Efficient polynomial multiplication can be done in NTT domain using point-wise multiplication by Equation 4.4. The individual coefficients of the product polynomial in NTT domain by Equation 4.5 and 4.6. For vector multiplication, it performs dot product on the vector by Equation 4.7.

$$\hat{f} \cdot \hat{g} = \hat{h} = (\hat{f}_{2i} + \hat{f}_{2i+1}X) \cdot (\hat{g}_{2i} + \hat{g}_{2i+1}X) \mod (X^2 - \zeta^{(2\text{BitRev}_7(i)+1)}) \tag{4.4}$$

$$\hat{h}_{2i} = \hat{f}_{2i}\hat{g}_{2i} + \hat{f}_{2i+1}\hat{g}_{2i+1}\zeta^{2\text{BitRev}_7(i)+1} \tag{4.5}$$

$$\hat{h}_{2i+1} = \hat{f}_{2i}\hat{g}_{2i+1} + \hat{f}_{2i+1}\hat{g}_{2i} \tag{4.6}$$

$$\mathbb{Z}_q^{256k} \cdot \mathbb{Z}_q^{256k} = \mathbb{Z}_q^{256k} = \sum_{i=0}^{2} \mathbb{Z}_q^{256}[i] \cdot \mathbb{Z}_q^{256}[i] \tag{4.7}$$

The module performing the polynomial vector multiplication is denoted as Vector Point-wise Multiplication. Its RAM architecture is shown in Figure 4.13. It performs dot product on the polynomial vector in RAM A and RAM B, and sum the intermediate product of polynomials to RAM C.

Hardware module for Point-wise multiplication is shown in Figure 4.14, it is pipelined so that $\hat{f}$ and $\hat{g}$ can be fed into this module one after another without delay. Note that Multiply and Reduce (a 32-bit multiplier and a Montgomery reduce) takes two cycles while the buffers (implemented by registers) takes one cycle to execute.

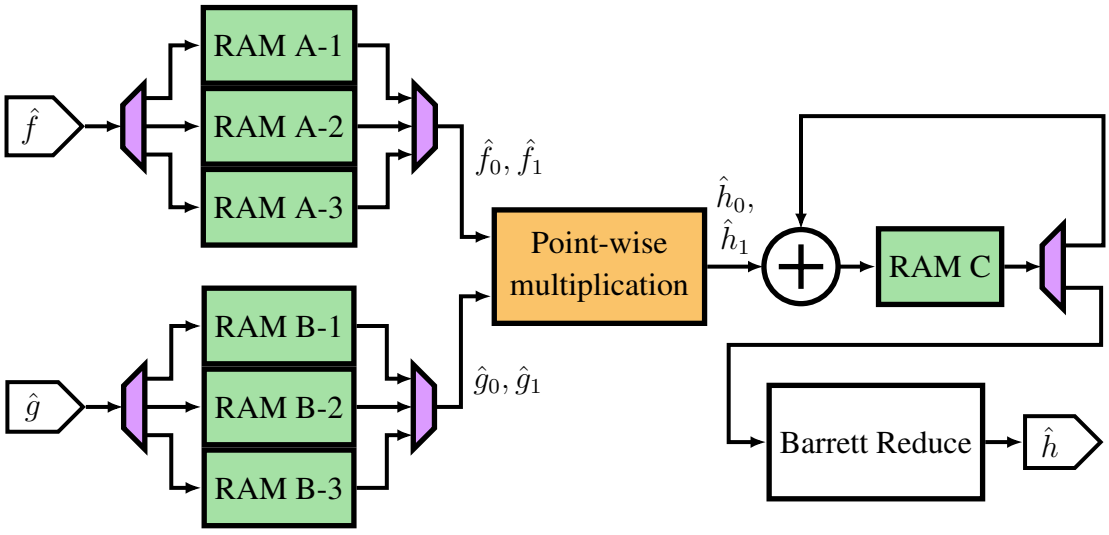


Figure 4.13: Vector Point-wise multiplication module's RAM architecture

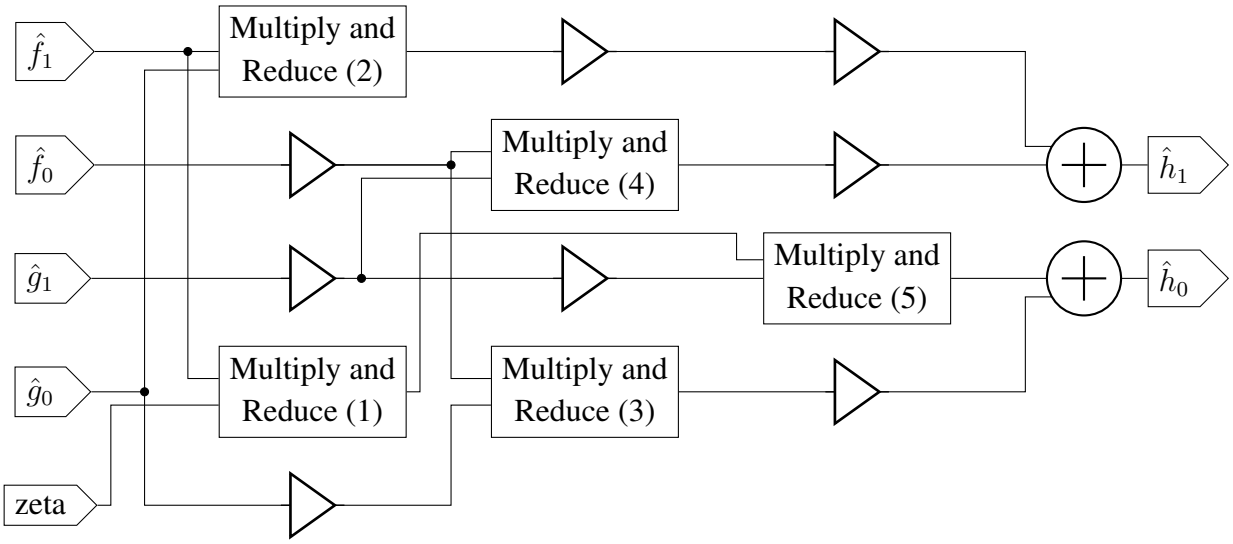


Figure 4.14: Point-wise multiplication hardware module

4.8 Decompress Module

Sine the secret key need to be converted in to a polynomial of zeros and ones then scaled by $q/2$, a decompression function is required. To protect the key from timing attack, the bit-scaling Algorithm 7 is implemented in constant-time.

This module erases the byte content after it's read out of the RAM block. The $i \in \{0, 1, 2, 3\}$ index in Figure 7 is dictated by a finite state machine that controls the timing.

Algorithm 7 Decompress of 1 bit (decompress_cal)

Input: bit b
Output: coefficient t
$t \leftarrow (b) + 1$
$t \leftarrow t \,\&\, 1665$
Return t

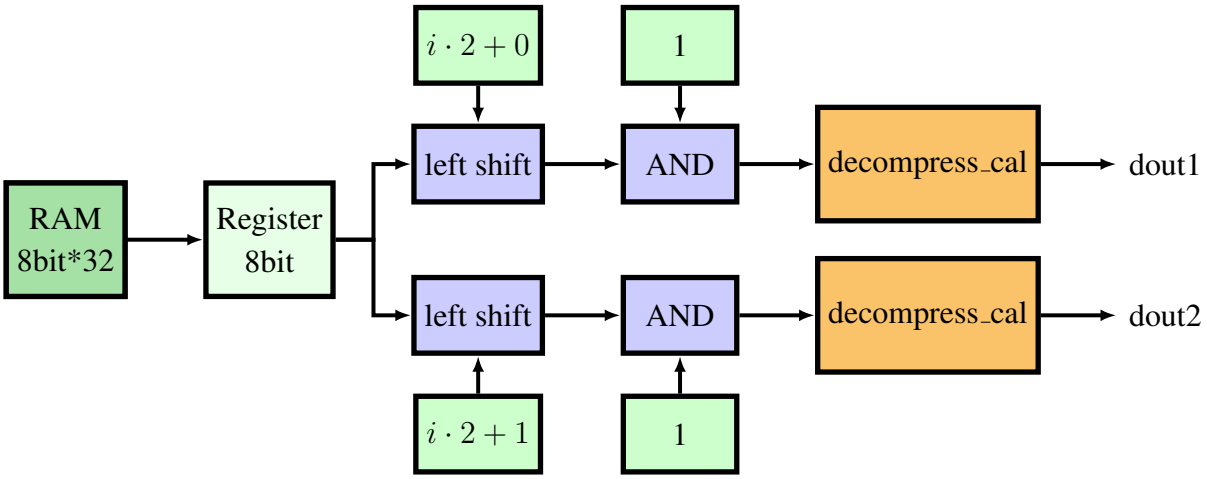


Figure 4.15: Decompress hardware module

4.9 Compress and Encode

Compression takes place to reduce the total ciphertext size without affecting the correctness of LWE, it is denoted in Equation 4.8. The division part is done with Division by Invariant Multiplication [10]. The algorithm implemented in hardware is the same as the one implemented in reference software implementation [3] but in a pipelined manner.

$$\lceil 2^d \cdot x/q \rfloor \mod 2^d \tag{4.8}$$

For encoding the compressed integers, only $d = 4$ and $d = 10$ are implemented for compressing v and $\mathbf{u}$. Encoding 4-bit integer into 16-bit array is trivial, since it naturally aligns without issue.

Encoding 10-bit integer into pipelined hardware is shown in Figure 4.16 as a timing graph. Each squares on the horizontal line is a 10-bit register and there are six of them in the module. Two 10-bit integers are inputted at the same time and shifted to the next register at the same time as shown in the figure. Each bracketed registers denote the 32-bit value being split in two to be outputted in 16-bit pairs. This particular design is chosen because of the fact that it can be pipelined and used continuously without the need to reset it in normal operation.

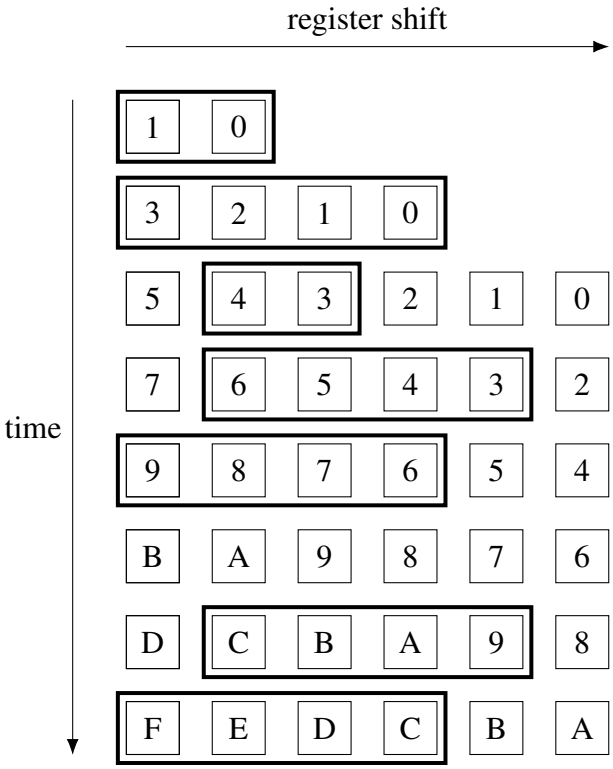


Figure 4.16: Encoding 10-bit integer into $16 \cdot 2$ array timing graph

4.10 Accumulator

We design accumulators placed in the very end of module architecture. Accumulator 1 contains $\mathbf{u}$ while Accumulator 2 contains v.

The pre-ciphertext are subsequently reduced via Barrett reduction, then compressed and encoded to reduce the ciphertext size. We integrate the compress and encode in the accumulator to reduce the complexity in high level consolidation.

Once $\mathbf{u} = \mathrm{NTT}^{-1}(\hat{\mathbf{A}} \circ \hat{\mathbf{y}}) + \mathbf{e_1}$ and $v = \mathrm{NTT}^{-1}(\hat{\mathbf{t}} \circ \hat{\mathbf{y}}) + e_2 + \mu$ are summed in its respective accumulators, their Small Controllers triggers the Compress and Encode (reduced) function. When the ciphertext is ready the module will send a `done` signal, and it will propagate to Small Controller and to the Big Controller.

4.11 Verification

We verify the correctness on this work using Python scripts in Jupyter Notebook. We translated the C reference implementation to Python to simplify the verification process. The experiment step can be reproduced as follows.

1. Generate test vector that is written to text files by Python scripts.
2. Run simulation until the ciphertext is outputted to testbench.
3. Issue command to ModelSim to save the ciphertext to a text file.
4. Run the verification Python script using the generated test vector and the ciphertext.

Chapter 5 Discussion

This chapter we address the design choices, trade-offs, and potential improvements.

Module characteristics: The cycle used for individual modules without accounting for the I/O cycles is shown in Table 5.1. Kyber encryption's total cycles from `reset` to `done` is 12391, which mostly consists of NTT, InvNTT and VPWM computation latency. It can be dramatically improved by increasing butterfly operations to two or more in NTT and InvNTT and increase the number of point-wise multiplication hardware. Such design require more sophisticated memory layout planning to achieve more parallelism without RAM read/write address conflicts.

NTT/InvNTT delay: The experiment shown as RAM wave graph in Figure 5.1 indicates that the high level module is mostly waiting for NTT to complete computation for $\hat{\mathbf{y}}$. Contributing some of the cycle for the overall cycle count.

Module	Cycles	Instance(s)	Used time(s)
NTT	940	1	3
InvNTT	940	1	4
VPWM	1072	1	4
CBD	37	1	7
Decompress	221	1	1
Compress and Encode	126	4	-
Kyber Encryption	12391	1	-

Table 5.1: Hardware module cycle count

Datapath and control separation: Some earlier modules such as CBD and NTT lack computation and control separation, so in NTT the index for RAM read/write are also the control for timing. This introduces the complication of making the HDL code harder to maintain and scale.

Security consideration: The security hardness of this work is yet to be addressed, as cryptographic hardware should be secure against side-channel and fault injection attacks. The operations for critical information such as keys are unmasked. Each module inherently trusts the input they are given without verification. So if one module glitches and produces undefined results, the subsequent modules will be given faulty input, ending up hindering the security of the encryption key.

Design robustness: While the trivial test vectors ($\{0, 1, 2, ..., n\}$) are not necessary legal inputs, the subsequent tests vectors generated are all legally generated via the key generation algorithm specified in FIPS203 [2]. Since it lacks verification, the robustness of the hardware implementation is yet to be address. However, the correctness is ensured.

Figure 5.1: ModelSim wave graph for all RAMs responsible for Kyber Encryption

References

[1] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS – kyber: a CCA-secure module-lattice-based KEM," Cryptology ePrint Archive, Paper 2017/634, 2017. [Online]. Available: https://eprint.iacr.org/2017/634

[2] "Module-lattice-based key-encapsulation mechanism standard," National Institute of Standards and Technology, Aug 2024. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[3] J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "pq-crystals kyber," 2024. [Online]. Available: https://github.com/pq-crystals/kyber

[4] "Sha-3 standard: Permutation-based hash and extendable-output functions," National Institute of Standards and Technology, Aug 2015. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.202

[5] "Sha-3 derived functions: cshake, kmac, tuplehash and parallelhash," National Institute of Standards and Technology, Dec 2016. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-185

[6] P. L. Montgomery., "Modular multiplication without trial division." 1985. [Online]. Available: http://www.ams.org/journals/mcom/1985-44-170/S0025-5718-1985-0777282-X/S0025-5718-1985-0777282-X.pdf

[7] P. Barrett, "Implementing the Rivest, Shamir and Adleman public key encryption algorithm on a standard digital signal processor," in *Advances in Cryptology — CRYPTO' 86*, A. M. Odlyzko, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1987, pp. 311–323. [Online]. Available: https://link.springer.com/chapter/10.1007/3-540-47721-7_24

[8] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex fourier series." *Mathematics of computation*, vol. 19, no. 90, pp. 297—-301, 1965. [Online]. Available: https://www.ams.org/journals/mcom/1965-19-090/S0025-5718-1965-0178586-1/S0025-5718-1965-0178586-1.pdf

[9] W. M. Gentleman and G. Sande, "Fast fourier transforms: for fun and profit," in *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference*, ser. AFIPS '66 (Fall). New York, NY, USA: Association for Computing Machinery, 1966, p. 563–578. [Online]. Available: https://doi.org/10.1145/1464291.1464352

[10] T. Granlund and P. L. Montgomery, "Division by invariant integers using multiplication," *SIGPLAN Not.*, vol. 29, no. 6, p. 61–72, Jun. 1994. [Online]. Available: https://doi.org/10.1145/773473.178249